

תהליך בניית אתר עם Supabase — מלא עם הסברים

=====

מדריך מפורט לכל השלבים, כולל הסבר על משמעות כל פעולה

שלב 0 — פתיחת הפרויקט והכנת הקבצים

לפני שמתחילים לבנות כלום, אנחנו צריכים לארגן לקלוד "תיק עבודות" — תיקיה מסודרת עם כל ההוראות שהוא יצטרך. ככה קלוד יודע מה בדיוק לבנות, באיזה סדר, ועם אילו פרטים.

שלב 0.1 — קבלת קובץ plan.md מהפרומפטר של Let's AI

הפרומפטר של Let's AI הוא כלי שמייצר עבורנו קובץ תכנון מסודר בשם plan.md. הקובץ הזה הוא "מפרט הפרויקט" — הוא מסביר לקלוד מה האתר צריך לכלול: אילו קטעים יש בדף, מה מטרת האתר, מי קהל היעד, מה הטקסטים, ואיזה פונקציות נדרשות.

בלי הקובץ הזה קלוד לא יודע מה לבנות — הוא יכול לנחש, אבל התוצאה תהיה גנרית. עם plan.md ברור ומפורט, קלוד יכול לבנות בדיוק את מה שרוצים.

מה עושים:

נכנסים לפרומפטר של Let's AI, ממלאים את כל פרטי הפרויקט (שם העסק, תחום, קהל יעד, שירותים), ומייצאים את הקובץ שנוצר.

שלב 0.2 — קבלת קובץ Phase 1

Phase 1 הוא קובץ הוראות טכניות לקלוד. בעוד ש-plan.md מסביר "מה" לבנות, קובץ Phase 1 מסביר "איך" לבנות — אילו קבצי קוד ליצור, איך לחבר את Supabase, איזו מבנה תיקיות להכין, ואיזה טבלאות לפתוח בבסיס הנתונים.

הקובץ הזה נכתב פעם אחת ומתאים לכל פרויקט בסגנון דומה. הוא חוסך מאיתנו את הצורך להסביר לקלוד את כל הפרטים הטכניים מחדש בכל פרויקט.

מה עושים:

מורידים את קובץ Phase 1 (קיים בחומרי הסדנה של Let's AI).

שלב 0.3 — קבלת קובץ Claude Design

קובץ Claude Design הוא הוראות עיצוב לקלוד — הוא מכיל את הצבעים של המותג, הפונטים, הסגנון הכללי, ולעיתים גם תמונות השראה. זה מה שגורם לאתר שקלוד יבנה להיראות כמו הפרויקט הספציפי שלנו ולא כמו אתר ברירת מחדל אפורי.

בלי הוראות עיצוב קלוד ישתמש בצבעים וסגנון אקראיים. עם קובץ עיצוב מסודר — האתר ייצא ברמת מקצועי.

מה עושים:

יוצרים קובץ עיצוב דרך הפרומפטר של Let's AI, או משתמשים בקובץ ברירת מחדל שמגיע עם חומרי הסדנה.

שלב 0.4 — יצירת תיקיית הפרויקט

קלוד עובד עם קבצים מקומיים — הוא צריך תיקיה ספציפית על המחשב שלנו כדי לשים בה את כל קבצי הפרויקט. זו תהיה "הבית" של הפרויקט שלנו.

מה עושים:

פותחים את File Explorer ומנווטים לנתיב:

C: → Users → [שם המשתמש שלכם] → projects. → claude

בתוך תיקיית projects יוצרים תיקיה חדשה עם שם הפרויקט — חשוב שהשם יהיה באנגלית, ללא רווחים, ללא תווים מיוחדים. לדוגמה: "my-business-site" או "yaels-bakery".

למה בדיוק שם? כי Claude Code יודע לחפש פרויקטים בתיקיית claude/projects ולהתחבר אליהם אוטומטית.

שלב 0.5 — העברת הקבצים לתיקיה

עכשיו מעבירים לתוך התיקיה החדשה שיצרנו את הקבצים שקיבלנו:

- plan.md

- Phase 1

- Claude Design (אם יש)

זה הכל. קלוד ידע לאתר אותם ולקרוא אותם כשנבקש ממנו.

שלב 0.6 — Plan Mode — קלוד קורא ומתכנן

זהו שלב קריטי שהרבה אנשים מדלגים עליו ומצטערים. לפני שקלוד מתחיל לבנות, אנחנו מבקשים ממנו להיכנס ל"מצב תכנון" — Plan Mode. במצב הזה קלוד קורא את כל הקבצים, מבין מה צריך לעשות, וכותב לעצמו תוכנית עבודה — אבל עדיין לא עושה כלום.

למה זה חשוב? כי בלי תכנון מוקדם, קלוד עלול לפתוח בכיוון לא נכון, לשכוח פרטים, או לבנות דברים בסדר לא הגיוני. עם Plan Mode הוא "חושב לפני שפועל".

מה עושים:

פותחים את Claude Desktop, מחברים אותו לתיקיה שיצרנו, ומבקשים ממנו לעבור ל- Plan Mode ולקרוא את plan.md ואת קובץ Phase 1. קלוד יכתוב תוכנית — קוראים אותה ובודקים שהיא הגיונית.

שלב 0.7 — Accept — אישור התוכנית

אחרי שקלוד כתב את תוכנית העבודה שלו וקראנו אותה — לוחצים Accept. זה האות לקלוד שאנחנו מאשרים לו לצאת לעבוד. הוא יתחיל ליצור את קבצי הפרויקט — Next.js, תיקיות, קבצי קוד ראשוניים.

בשלב זה קלוד יוצר את שלד הפרויקט הריק. הוא לא מחובר עדיין לסופהבייס — זה יקרה בשלבים הבאים.

שלב 1א — הקמת פרויקט Supabase

Supabase הוא שירות בסיס נתונים שעובד בענן — הוא זה שישמור לנו את כל הלידים, הנתונים מהטפסים, ומידע כל מה שהמשתמשים ימלאו באתר. בגלל שקלוד צריך לדעת לאן "לשלוח" את הנתונים, אנחנו פותחים את פרויקט Supabase לפני שהוא ממשיך לבנות.

Supabase הוא חינמי לפרויקטים קטנים ובינוניים, ועובד מצוין עם Next.js.

שלב 1א1 — הרשמה ל-Supabase

גולשים לאתר supabase.com ולוחצים על "Start your project". אם זו הפעם הראשונה — מתחברים עם חשבון Google (הדרך הכי מהירה). Supabase יפתח את ה-Dashboard — הממשק המרכזי שדרכו נשלט על כל בסיסי הנתונים שלנו.

שלב 2.א1 — יצירת פרויקט חדש

בתוך ה-Dashboard לוחצים על הכפתור הירוק "New project". מופיע טופס ובו ממלאים:

שם הפרויקט (Name) — שם תיאורי באנגלית, ללא רווחים. זה השם שיזהה את בסיס הנתונים שלנו.

סימט מסד הנתונים Supabase — (Database Password) מבקש סימט חזקה להגנה על בסיס הנתונים. חשוב מאוד: רושמים את הסימט במקום בטוח! אי אפשר לשחזר אותה לאחר מכן, רק לאפס.

Region — המיקום הגיאוגרפי של השרת. ברירת המחדל בסדר גמור — לא חייבים לשנות.

לאחר מכן לוחצים "Create new project" וממתינים כ-30 שניות עד שהפרויקט נוצר.

שלב 3.א1 — העתקת Project ID

ה-Project ID הוא המספר הייחודי של הפרויקט שלנו ב-Supabase. נצטרך אותו כשנחבר את ה-MCP Connector — שהוא הגשר שמאפשר לקלוד לדבר ישירות עם בסיס הנתונים.

מה עושים:

בתפריט הצד לוחצים על Settings ואז General. בחלק "General Settings" מוצאים את שדה "Project ID" ולוחצים על כפתור ה-Copy שלצידו. שומרים את המחרוזת בצד — בפתקית, בנוטפד, בכל מקום שנוח.

שלב 4.א1 — העתקת Project URL ו-API Keys

עכשיו בתפריט הצד לוחצים על Settings ואז API Keys. כאן נמצאים שלושת הפרטים שנצטרך לקובץ הסיסמאות (env.local):

Project URL — הכתובת של בסיס הנתונים שלנו. נראית כך:
`https://xxxxxxxxxx.supabase.co`

זו הכתובת שדרכה הקוד מתחבר לסופהבייס.

Publishable key — מפתח API ציבורי שמאפשר לקוד שלנו לשלוח נתונים לסופהבייס. מתחיל ב-`_sb_publishable`. מפתח זה בטוח להכניס לקוד כי הוא מוגבל בהרשאות.

Secret key — מפתח סודי עם הרשאות מלאות לבסיס הנתונים. מתחיל ב-`_sb_secret`. לעולם לא שמים אותו בקוד שעולה לאינטרנט — הוא מיועד רק לשרתים ותהליכים אוטומטיים. אם הוא מגיע לידיים לא נכונות, מישהו יכול לעשות כל דבר בבסיס הנתונים שלנו.

מעתיקים את שלושתם ושומרים אותם בצד.

שלב 1ב — הכנסת פרטי החיבור לפרויקט (env.local)

עכשיו שיש לנו את כל הפרטים של Supabase, צריך "להגיד" לקוד שלנו איך להתחבר לשם. עושים זאת דרך קובץ מיוחד בשם `env.local` — קובץ "סביבתי" שמכיל משתנים סודיים.

מה זה `env.local` ולמה הוא חשוב?

בניגוד לשאר קבצי הקוד, קובץ `env.local` לא עולה לאינטרנט ולא מועלה ל-GitHub. הוא קיים רק על המחשב המקומי שלנו. זה המקום הנכון לשים בו פרטים סודיים כמו כתובות בסיס נתונים ומפתחות API — כי הם לא אמורים להיחשף לעולם.

קלוד יצר את הקובץ הזה ריק כשבנה את שלד הפרויקט. עכשיו אנחנו ממלאים אותו.

שלב 1.1 — פתיחת הקובץ

פותחים את סייר הקבצים, נכנסים לתיקיית הפרויקט שקלוד יצר, ומחפשים את הקובץ `env.local` בשורש התיקה (לא בתיקייה פנימית — ממש בשורש). פותחים אותו עם VS Code או Cursor. זו הפעם היחידה שפותחים כלי עריכה בכל התהליך הזה.

שלב 2.1 — מילוי הפרטים

הדביקו את שלוש השורות הבאות בדיוק — כל שורה בשורה נפרדת, עם Enter אחרי כל שורה:

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
..._NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_pUBLISHABLE
NEXT_PUBLIC_SUPABASE_PROJECT_ID=xxxxxxxxxxxxxxxxxxxxxx
```

כמה כללים חשובים:

— אין רווח לפני ואחרי סימן ה-`=` (שוויון)

— כל שורה בשורה נפרדת

— אין גרשיים ("") סביב הערך

— מחליפים את ה-`xxxx` בערכים האמיתיים שהועתקו מסופהבייס

שלב 3.ב1 — שמירת הקובץ

שומרים עם Ctrl+S. ברגע השמירה, הפרויקט "יודע" כיצד להתחבר לסופהבייס.

שלב 1ג — חיבור ה-Supabase MCP Connector

עד עכשיו הכנו הכל — אבל קלוד עצמו עדיין לא יכול לגעת בבסיס הנתונים. ה-MCP Connector הוא הגשר שמחבר בין Claude Desktop לבין פרויקט Supabase שלנו.

מה זה MCP Connector ולמה הוא נחוץ?

MCP (Model Context Protocol) הוא מנגנון שמאפשר לקלוד לתקשר עם שירותים חיצוניים — כמו Supabase. בלי ה-Connector, קלוד יכול לכתוב קוד שמדבר עם Supabase, אבל הוא לא יכול להריץ פקודות ישירות בבסיס הנתונים. עם ה-Connector — קלוד יכול ליצור טבלאות, להכניס נתונים לדוגמה, לבצע שאילתות SQL, ולבדוק שהכל עובד.

זה מה שגורם לתהליך לעבוד בצורה אוטומטית לגמרי — אנחנו לא צריכים להריץ SQL ידנית. קלוד עושה הכל בשבילנו.

שלב 1ג1 — כניסה להגדרות Connectors

ב-Claude Desktop לוחצים על "Customize" בפינה ומבחרים ב-"Connectors". כאן מופיעה רשימה של כל החיבורים הזמינים לקלוד.

שלב 2.g1 — חיבור Supabase

מחפשים את Supabase ברשימה ולוחצים לחבר. Supabase יבקש את ה-Project ID שהועתק בשלב 1א. מצינים אותו ומאשרים.

כשהחיבור מצליח — רואים ש-Supabase מסומן כ"מחובר". מעכשיו קלוד יכול לפעול ישירות בבסיס הנתונים.

שלב 3.g1 — אימות החיבור

לפני שממשיכים, כדאי לוודא שהחיבור עובד. אפשר לשאול את קלוד "האם אתה מחובר ל-Supabase?" — הוא יאשר ויציג את שם הפרויקט המחובר.

שלב 1ד — קלוד בונה את תשתית הקוד

עכשיו שיש לנו פרויקט Supabase פתוח, פרטי חיבור בקובץ .env.local, וה-MCP Connector מחובר — קלוד יכול להתחיל לבנות את החלק הטכני. זהו שלב שבו קלוד עובד לבד וצריך רק להמתין.

שלב 1.ד1 — מסירת Phase 1 לקלוד

מעבירים לקלוד את קובץ Phase 1 ומבקשים ממנו לבצע אותו. קלוד יקרא את ההוראות הטכניות ויתחיל לעבוד.

שלב 2.ד1 — קלוד מתקין את חבילת Supabase

קלוד מריץ את הפקודה:

`npm install @supabase/supabase-js`

זוהי חבילת הקוד הרשמית של Supabase עבור JavaScript/TypeScript. היא מכילה את כל הפונקציות שמאפשרות לנו לשלוח ולקבל נתונים מבסיס הנתונים.

שלב 3.ד1 — קלוד יוצר קבצי קוד

קלוד יוצר מספר קבצים חשובים:

`lib/supabase/client.ts` — קובץ שיוצר "חיבור פתוח" לסופהבייס, שכל שאר הקוד ישתמש בו.

`lib/leads/submitLead.ts` — הפונקציה שמקבלת את הנתונים מהטופס ושולחת אותם לבסיס הנתונים. זו הלוגיקה שרצה כשמישהו לוחץ "שלח" בטופס שלנו.

`supabase/schema.sql` — הגדרת מבנה הטבלאות — אילו עמודות יש לטבלת הלידים, מה סוג כל שדה (טקסט, מספר, תאריך...), ואיזה שדות הם חובה.

`env.example` — עותק ריק של קובץ `env.local`, ללא ערכים אמיתיים. זה קובץ שכן יכול לעלות ל-GitHub, כדי שאנשים אחרים שיורידו את הפרויקט ידעו מה ממלאים.

שלב 4.ד1 — קלוד יוצר את הטבלאות בסופהבייס

זה הרגע שבו ה-MCP Connector "עובד". קלוד מריץ את קובץ ה-SQL ישירות דרך החיבור לסופהבייס — ויוצר את הטבלאות ממש בבסיס הנתונים. לא צריך לעשות כלום — ממתינים.

כשקלוד מסיים, ניתן לגשת ל-Supabase Dashboard ב-Table Editor ולראות שהטבלאות נוצרו.

בדיקת אימות: לכו ל-Table Editor → Supabase Dashboard. אם הטבלאות מופיעות — הכל עבד!

שלב 2 — בניית ממשק המשתמש

עכשיו כשהתשתית הטכנית מוכנה — מגיע השלב הכיפי. נותנים לקלוד את חומרי העיצוב ואת תמונות הפרויקט, ומבקשים ממנו לבנות את האתר הנראה.

שלב 2.1 — הכנסת תמונות לפרויקט

Next.js מכיל תיקיה בשם public/ בשורש הפרויקט. כל תמונה שנשים שם תהיה נגישה מהדפדפן. אנחנו יוצרים בתוכה תיקיה assets/ ושמים שם את כל התמונות.

איזה תמונות מכינים:

— לוגו — PNG עם רקע שקוף, שם הקובץ: logo.png

— תמונת Hero — תמונה גדולה ואיכותית לחלק העליון של הדף, שם: hero.jpg

— תמונת אודות — תמונה של בעל/ת העסק, שם: about.jpg

— תמונות שירות/מוצר — תמונה לכל שירות או מוצר שמוצג באתר, שמות: product-1.jpg, product-2.jpg וכו'

חשוב: כל שמות הקבצים באנגלית, ללא רווחים.

שלב 2.2 — מסירת קובץ Claude Design לקלוד

נותנים לקלוד את קובץ העיצוב — Claude Design command. הקובץ מכיל את הצבעים, הפונטים, הסגנון הכללי, ואפילו טקסטים לדוגמה. קלוד ישתמש בכל אלה כדי לבנות את הממשק.

אפשר גם לצרף תמונת השראה — לדוגמה, צילום מסך של אתר שמוצא חן בעיניכם. קלוד יבין מאיזה כיוון עיצובי לצאת.

שלב 2.3 — מסירת Phase 2 לקלוד

נותנים לקלוד את קובץ Phase 2 ומבקשים ממנו לבצע אותו. Phase 2 מכיל את ההוראות לבניית הממשק עצמו — אילו קומפוננטות ליצור ואיך לחבר אותן.

שלב 2.4 — קלוד בונה את האתר

קלוד בונה את כל חלקי הדף לפי העיצוב שנתנו:

ניווט עליון (Header) — הלוגו ותפריט קישורים

סקשן פתיח (Hero) — תמונה גדולה עם כותרת ראשית וכפתור קריאה לפעולה

סקשן אודות — הצגת בעל/ת העסק או החברה

סקשן שירותים/מוצרים — רשת קומפוננטות עם תמונות ותיאורים

סקשן קהל יעד — למי מיועד השירות

טופס לידיים — שם, טלפון, אי-מייל, הסכמה — עם כפתור שליחה

כותרת תחתונה (Footer) — פרטי קשר וקישורים

קלוד גם מחבר את הטופס לפונקציית submitLead שנוצרה ב-Phase 1 — כלומר שליחה מהטופס תישמר אוטומטית בסופהבייס.

שלב 2.5 — קלוד מפעיל שרת פיתוח

קלוד מריץ:

```
npm run dev
```

האתר נפתח בדפדפן בכתובת localhost:3000. אפשר לראות אותו ולעשות תיקונים.

שלב 2.6 — בדיקה ושיפורים

עכשיו מסתכלים על האתר ומבקשים מקלוד שינויים לפי הצורך:

— שינויי צבעים או פונטים

— אנימציות ותנועה בכניסת אלמנטים

— שינויי מרווחים ופריסה

— הוספת אלמנטים חדשים

— שיפורי מובייל

אין גבול — אפשר לבקש כמה תיקונים שרוצים עד שהאתר נראה בדיוק כמו שרוצים.

שלב 2.7 — בדיקת הטופס מקצה לקצה

זהו שלב קריטי — לא מסיימים עד שעושים בדיקה מלאה:

ממלאים שם, טלפון, ו/או אי-מייל בטופס ולוחצים שלח.

רואים הודעת הצלחה — הנתונים נשלחו.

נכנסים ל-Table Editor → Supabase Dashboard ובודקים שהשורה החדשה מופיעה בטבלה.

אם הנתון הגיע — הכל עובד! האתר חי ומחובר לבסיס נתונים אמיתי.

סיכום — מה בנינו?

בסיום התהליך יש לנו:

אתר Next.js מקצועי ומעוצב לפי המותג שלנו — עם ניווט, Hero, אודות, שירותים, וטופס.

בסיס נתונים Supabase חי בענן — שמקבל ושומר כל ליד שנשלח מהאתר.

חיבור מלא בין הטופס לבסיס הנתונים — כל פעם שמישהו מגיש טופס, הנתונים נשמרים אוטומטית.

תשתית מוכנה לעלות לאוויר — הפרויקט מוכן לפריסה ב-Vercel או כל פלטפורמת Hosting אחרת.

האתר הוא שלנו, בשרת שלנו, עם קוד שקלוד כתב ואנחנו מבינים — לא פלטפורמת אתרים מוגבלת.

צ'קליסט — לוודא שהכל הושלם

- [] פרויקט Next.js נוצר בתיקיה הנכונה
- [] plan.md, Phase 1 ו-Claude Design נמצאים בתיקיה
- [] קלוד אישר Plan Mode ויצא לעבוד
- [] פרויקט Supabase נפתח עם שם וסיסמה
- [] Project ID, Project URL ו-Publishable key הועתקו ונשמרו
- [] קובץ env.local מולא בדיוק ונשמר עם Ctrl+S
- [] MCP Connector של Supabase מחובר ב-Claude Desktop
- [] קלוד יצר את קבצי Phase 1
- [] הטבלאות קיימות ב-Table Editor → Supabase Dashboard
- [] התמונות והלוגו נמצאים בתיקיית /public/assets
- [] קלוד בנה את כל קומפוננטות ה-UI
- [] האתר פתוח ב-localhost:3000
- [] טופס הלידים עובד ושומר נתונים בסופהבייס